

DOCUMENTATION

SCRIPT-CONTROLLED ACL - RESTRICT RECORD ACCESS BASED ON FIELD VALUE

1. Introduction

1.1 Project Overview

The project “Script-Controlled ACL – Restrict Record Access Based on Field Value” implements row-level, script-controlled Access Control Lists (ACLs) in ServiceNow to restrict record visibility and modification rights on a custom Institution Details table, based on the Branch field value and the user's role.

1.2 Purpose

The main purpose is to ensure that users see and manage only the records relevant to their branch (EEE), while administrators retain unrestricted access, enforcing data confidentiality and role-based governance without relying on client-side restrictions.

2. Ideation Phase

2.1 Problem Statement

Institution records (student, faculty, and branch details) stored in a single custom table are visible to all users by default. Without record-level restriction, users from one branch (e.g. ECE, CSE) could view or modify records belonging to another branch (e.g. EEE), creating a data privacy and governance risk.

2.2 Empathy Map Canvas

The main users are branch staff/faculty (e.g. EEE User) and System Administrators. Branch users expect to see only records relevant to their own branch without confusion or clutter, while administrators need full visibility and control to manage all records and resolve issues across branches.

2.3 Brainstorming

Proposed ideas include role-based Access Control Lists, script-controlled row-level filtering using Data Conditions, separate roles for Read/Create/Write/Delete operations, an admin-override check, and elevated-privilege ACL configuration.

3. Requirement Analysis

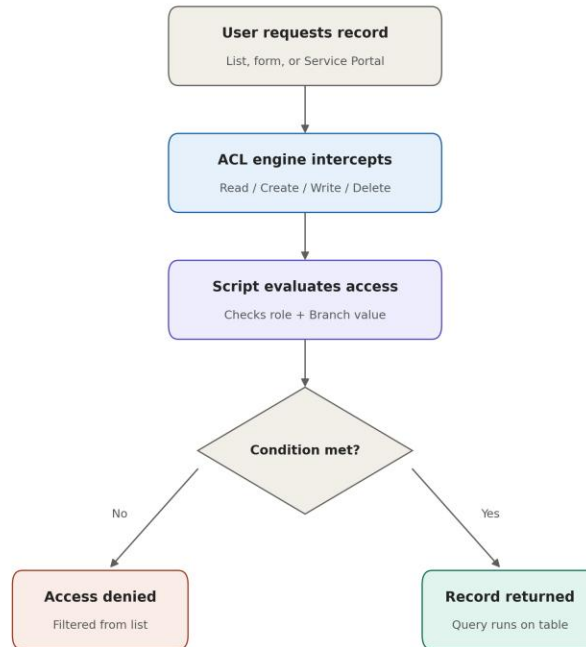
3.1 Customer Journey Map

The journey covers two personas — an EEE Branch User (bb1 role) who logs in, opens the Institution Details list, and sees only EEE-branch records because the script-controlled Read ACL evaluates role + Branch condition; and a System Administrator who elevates to security_admin and, since the ACL script checks gs.hasRole('admin') first, retains full Read/Create/Write/Delete access across all branches (ECE, EEE, CSE).

3.2 Solution Requirement

The system should restrict Read access to EEE branch records to users holding the bb1 role, restrict Create/Write/Delete to users holding the bb2/bb3/bb4 roles respectively, allow admin users to bypass all restrictions, enforce every check via server-side script rather than client-side logic, and ensure restricted records are simply excluded from view rather than shown with an error message.

3.3 Data Flow Diagram



User requests record → ACL engine intercepts (Read/Create/Write/Delete) → Script evaluates access (checks role + Branch value) → Condition met? → Yes: record returned / No: access denied (filtered from list).

3.4 Technology Stack

Component	Technology / Purpose
ServiceNow	Now Platform (Global scope) - core application, tables, roles, and ACL engine
Custom Table	u_institution_details - stores Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, Description
Script-Controlled ACLs	Record-level Read / Create / Write / Delete ACLs with Advanced scripting
Server-side Scripting	GlideSystem API - gs.hasRole('admin'), gs.hasRole('bb1'), etc.
Roles	admin, bb1 (Read), bb2 (Create), bb3 (Write), bb4 (Delete)
Data Condition	Branch is EEE, applied on the Read ACL
Update Set	ACL Project [Global] - packages all configuration changes

4. Project Design

4.1 Problem–Solution Fit

Uncontrolled, table-wide visibility is replaced with row-level, script-controlled ACLs that evaluate both the user's role and the record's Branch field before granting access — closing the gap between “any authenticated user can see everything” and “only the right branch sees the right data.”

4.2 Proposed Solution

The proposed solution defines four custom roles mapped to the four CRUD operations, and layers a Data Condition (Branch is EEE) on top of the Read ACL:

- bb1 → Read access, restricted to Branch = EEE records
- bb2 → Create access on the table
- bb3 → Write access on the table
- bb4 → Delete access on the table
- admin → Full access to all records regardless of branch (via `gs.hasRole('admin')` short-circuit)

4.3 Solution Architecture

EEE User / Admin → ServiceNow List / Form / Service Portal → ACL Engine (Read/Create/Write/Delete interception) → Script Evaluation (Role check + Data Condition) → Institution Details Table (`u_institution_details`). The custom table stores Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, and Description.

5. Project Planning & Scheduling

5.1 Project Planning

The project is planned through the following activities:

1. User and Role Creation (EEE User, bb1–bb4 roles)
2. Custom Table Creation (`u_institution_details`) and Field Configuration
3. Test Record Creation across ECE, EEE, and CSE branches
4. Read ACL Configuration (Data Condition + Script)
5. Create ACL Configuration
6. Write ACL Configuration
7. Delete ACL Configuration
8. Testing and Verification (via Impersonation)
9. Documentation and Deployment

The project uses an Update Set named “ACL Project [Global]” to manage the configuration.

6. Functional and Performance Testing

6.1 Functional Testing

Test Case	Input / Action	Expected Result
TC01	EEE User (bb1 role) opens Institution Details list	Only EEE branch records (2 records) displayed
TC02	User with no bb role opens the list	No records displayed
TC03	Admin opens the list	All records across ECE, EEE, CSE displayed
TC04	User with bb1 + bb2 roles clicks New	New button available; record created

Test Case	Input / Action	Expected Result
TC05	User with bb1 + bb2 + bb3 roles edits a field	EEE record editable inline
TC06	User with bb1-bb4 roles deletes a record	EEE record deleted successfully

6.2 Performance Testing

The ACL scripts are checked to ensure row-level filtering executes without noticeable delay on list load, and that the correct record set (filtered or full) is returned consistently on every refresh, impersonation switch, and CRUD action.

7. Results

7.1 Output Screenshots

The developed system successfully demonstrates:

- EEE branch user creation and role assignment
- Custom table (u_institution_details) with branch-wise records
- Read ACL restricting list view to EEE-only records for bb1 users
- Create, Write, and Delete ACLs enabling New/Edit/Delete actions only for authorized roles
- Admin override showing full, unrestricted access across all branches

8. Advantages & Disadvantages

8.1 Advantages

- Enforces true row-level, server-side security that cannot be bypassed from the client
- Keeps branch data private without duplicating tables per branch
- Granular control — separate roles for Read, Create, Write, and Delete
- Admin retains full oversight without extra configuration
- Reusable pattern — the same script logic can be extended to ECE/CSE branches

8.2 Disadvantages

- Requires careful role assignment; a missing role silently hides all records with no error message
- The Data Condition value (Branch = EEE) is hardcoded — supporting more branches needs additional ACLs or script updates
- Admin override bypasses all conditions, so misuse of elevated privileges carries a wide impact

9. Conclusion

The Script-Controlled ACL – Restrict Record Access Based on Field Value project provides a practical, row-level security solution for ServiceNow's Institution Details table. It ensures that EEE-branch users see and manage only their own branch's records, while administrators retain complete access — demonstrating how role checks and data conditions can be combined in server-side scripts to enforce fine-grained, non-bypassable access control. The approach can be extended to other branches and tables in future phases.